IN4640 Assignment 1 on Intensity Transformations and Neighborhood Filtering

Name: W.A.N.C. Wickramarathna

Index Number: 258844D

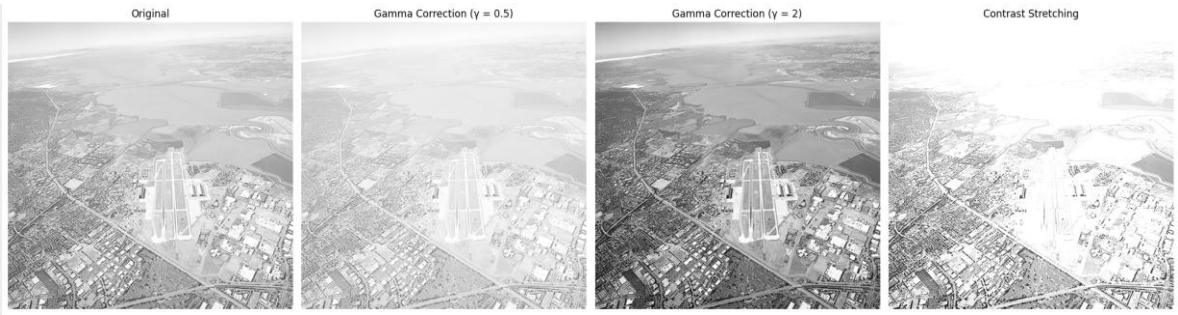
Github URL: https://github.com/nishanc/Computer-Vision-A1

1–3. Intensity Transformations and Histogram Equalization

The runway image was processed using gamma correction, contrast stretching and manual histogram equalization.

```
# Gamma correction and contrast stretching
img_float = img_rgb.astype(np.float32)/255.0
gamma_05 = img_float**0.5
gamma_2 = img_float**2
r1, r2 = 0.2, 0.8
contrast = np.where(img_float<r1,0,
           np.where(img_float>r2,1,(img_float-r1)/(r2-r1)))

# Manual histogram equalization
gray = cv.cvtColor(img_rgb, cv.COLOR_RGB2GRAY)
hist = np.bincount(gray.flatten(), minlength=256)
cdf = hist.cumsum()
cdf_norm = ((cdf-cdf.min())*255/(cdf.max()-cdf.min())).astype(np.uint8)
equalized = cdf_norm[gray]
```

- γ = 0.5 brightened the image, which made darker areas more visible.
- γ = 2 darkened the image, which suppressed highlights and increased shadow.
- Contrast stretching increased the range of mid-level intensities and improved overall contrast.
- Histogram equalization redistributed intensities and made features in the runway image clearer.



Q1: Original image, γ = 0.5, γ = 2 and contrast-stretching result.

Q3: Original grayscale runway image, equalized image and histogram(s).

4. Gamma Correction in L*a*b* Space and Otsu Thresholding

A gamma correction was applied only to the L (lightness) channel of the L*a*b* image to preserve color.

```
img_lab = cv.cvtColor(img_rgb, cv.COLOR_RGB2LAB).astype(np.float32)
L, a, b = cv.split(img_lab)
gamma = 0.6
L_corr = np.uint8(np.clip((L/255.0)**gamma * 255, 0, 255))
img_corr = cv.cvtColor(cv.merge([L_corr, a.astype(np.uint8), b.astype(np.uint8)]),
                       cv.COLOR_LAB2RGB)
```

- $\gamma = 0.6$ brightened the shadow regions while preserving natural color.
- The L histogram shifted toward higher intensities, showing that the corrected image became brighter.

For the "woman at the window" image, Otsu thresholding was used to separate the dark foreground (woman + room) from the bright outdoor background.

```
gray = cv.cvtColor(img_rgb, cv.COLOR_RGB2GRAY)
th, mask = cv.threshold(gray, 0, 255, cv.THRESH_BINARY_INV + cv.THRESH_OTSU)
# Otsu threshold = 101
```

- The Otsu threshold value was 101.
- Foreground-only histogram equalization revealed the woman's clothing, body outline, window structure and some interior details.
- Noise became more visible because very dark pixels were strongly stretched.

*Q2: Original and γ = 0.6 corrected image in L*a*b* space with histograms.*

Q4: Original grayscale image, Otsu mask (threshold = 101) and equalized foreground result.

5–6. Gaussian Filtering and Derivative of Gaussian

A normalized 5×5 Gaussian kernel with σ = 2 was computed in NumPy, visualized as a 51×51 3D surface and used to smooth the Einstein image. A derivative-of-Gaussian (DoG) kernel was also derived and applied in x- and y-directions.

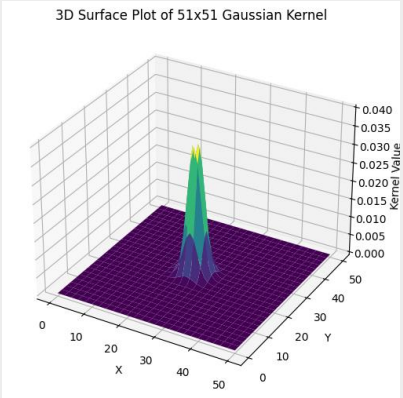
```
def gaussian_kernel(size, sigma):
    k = size//2
    x, y = np.meshgrid(np.arange(-k,k+1), np.arange(-k,k+1))
    K = np.exp(-(x**2+y**2)/(2*sigma**2))
    return K/np.sum(K)

def dog_kernels(size, sigma):
    k = size//2
    x, y = np.meshgrid(np.arange(-k,k+1), np.arange(-k,k+1))
    G = (1/(2*np.pi*sigma**2))*np.exp(-(x**2+y**2)/(2*sigma**2))
    Gx = -(x/sigma**2) * G;   Gy = -(y/sigma**2) * G
    return Gx/np.sum(np.abs(Gx)), Gy/np.sum(np.abs(Gy))
```

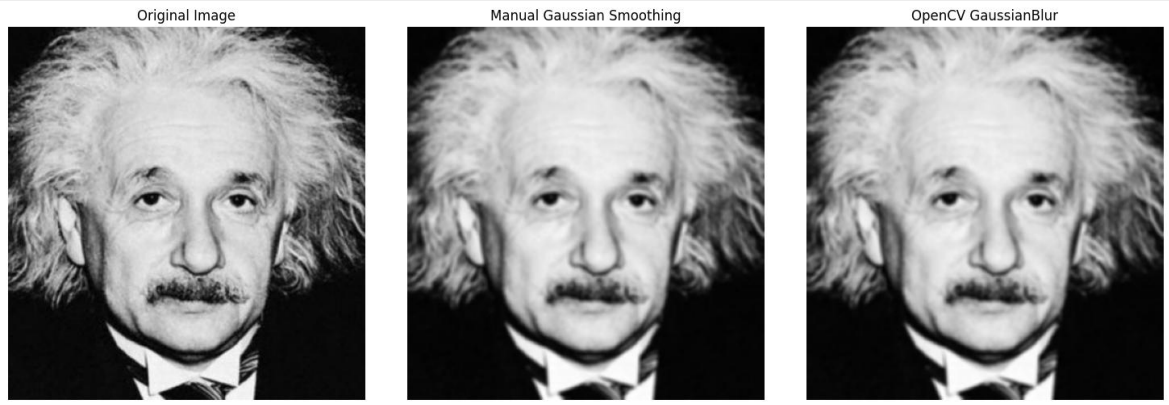
- Gaussian filtering reduced noise but also blurred some details. The OpenCV GaussianBlur result was very similar to the manual implementation.
- The DoG kernels highlighted changes in intensity. The x-direction emphasized vertical edges and the y-direction emphasized horizontal edges.

- Compared with the manual DoG, Sobel also detected edges but with slightly stronger/sharper responses.

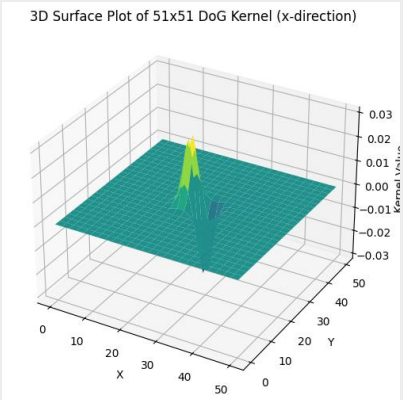
For the Gaussian function $G(x,y) = (1/(2\pi\sigma^2)) \exp(-(x^2+y^2)/(2\sigma^2))$, the first-order derivatives are:

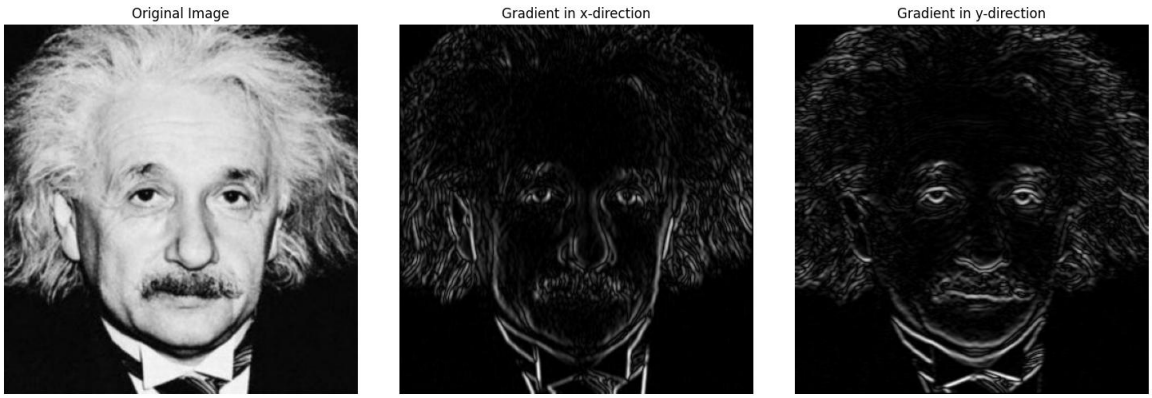
```
∂G/∂x = -(x/σ²) G(x,y)
∂G/∂y = -(y/σ²) G(x,y)
```





Q5: 51×51 3D Gaussian surface, original Einstein image and smoothed image (manual and OpenCV).





Q6: 5×5 derivative-of-Gaussian kernels in x and y, 51×51 3D surface (one direction), x/y gradients and comparison with Sobel.

7. Image Zooming Using Nearest Neighbor and Bilinear Interpolation

Two custom zoom functions were written to enlarge images by a factor s. The normalized sum of squared difference (SSD) was computed between the scaled-up small image and the corresponding original large image.

```
def normalized_ssd(img1, img2):
    a = img1.astype(np.float32)/255.0
    b = img2.astype(np.float32)/255.0
    return np.mean((a-b)**2)

zoom_nn = zoom_nearest(small, s, out_shape=large.shape[:2])
zoom_bl = zoom_bilinear(small, s, out_shape=large.shape[:2])
```

Image pair	Nearest neighbor SSD	Bilinear SSD
im01	0.0020956409	0.0017704936
im02	0.0004067065	0.00028291458
im03	0.0011250711	0.0007830615
taylor	0.0036605634	0.0034191327

- All normalized SSD values were small, so the enlarged images were close to the original images.
- For all four image pairs, bilinear interpolation gave smaller SSD values than nearest neighbor.
- Bilinear interpolation therefore produced smoother and more accurate zoomed images, while nearest neighbor caused more blockiness.

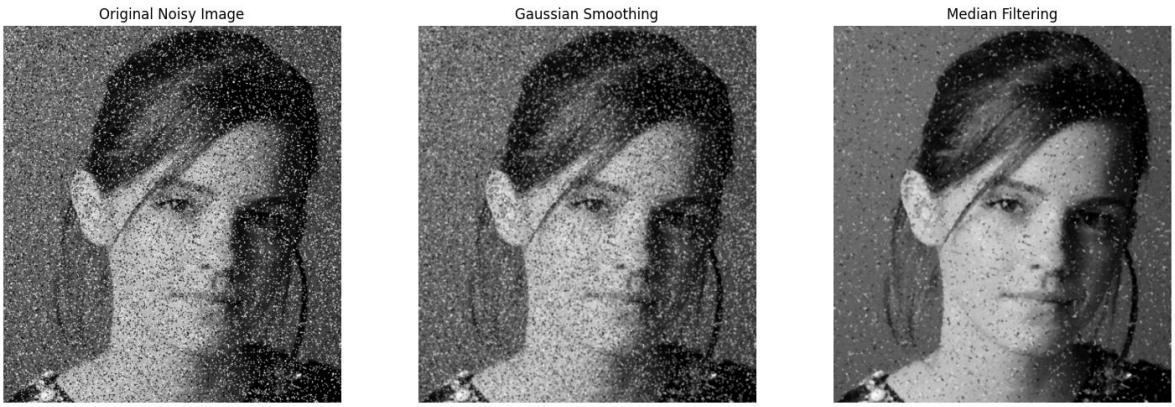
Q7: Examples showing the original large image, small image, zoomed nearest-neighbor result and zoomed bilinear result.

8–9. Salt-and-Pepper Noise Removal and Image Sharpening

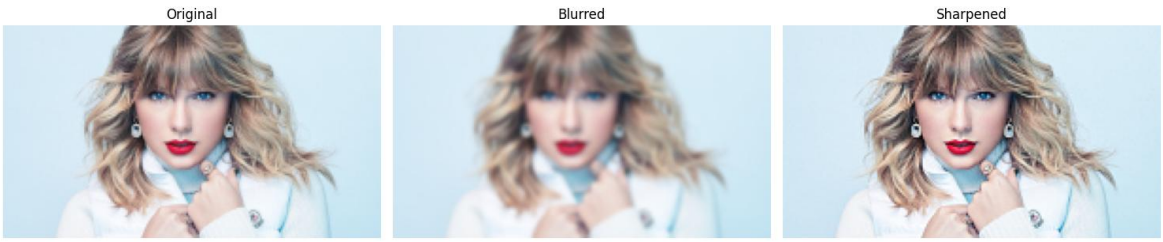
A noisy face image was processed using Gaussian smoothing and median filtering. Image sharpening was then demonstrated on taylor_small.jpg

```
gaussian = cv.GaussianBlur(noisy, (5,5), 1)
median = cv.medianBlur(noisy, 5)

# Unsharp masking
blurred = cv.GaussianBlur(img_rgb, (5,5), 1.0)
sharpened = cv.addWeighted(img_rgb, 1.5, blurred, -0.5, 0)
```

- Gaussian smoothing reduced some noise but blurred the face.
- Median filtering removed salt-and-pepper noise more effectively and preserved facial edges better.
- Sharpening improved visibility of hair, facial features and clothing edges in taylor_small.jpg.



Q8: Noisy image, Gaussian smoothing result and median filtering result.



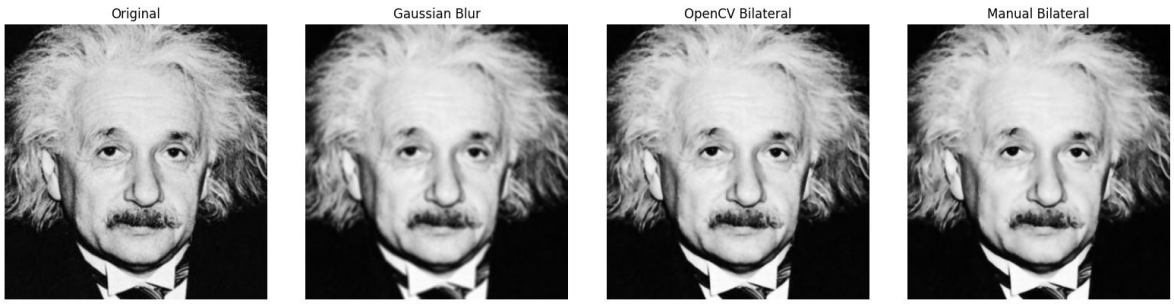
Q9: Original taylor_small.jpg, blurred version (optional) and sharpened result.

10. Bilateral Filtering

A bilateral filter was implemented manually and applied to Einstein. Results were compared with Gaussian smoothing and OpenCV's bilateral filter.

```
def manual_bilateral_filter(image, diameter, sigma_s, sigma_r):
    padded = np.pad(image.astype(np.float32), ((r,r),(r,r)), mode='reflect')
    # spatial Gaussian depends on distance
    # range Gaussian depends on intensity difference
    # output(i,j) = Σ(weights*region) / Σ(weights)

gaussian_img = cv.GaussianBlur(img, (5, 5), 2)
bilateral_cv = cv.bilateralFilter(img, d=5, sigmaColor=30, sigmaSpace=2)
```

- Gaussian smoothing blurred edges together with noise.
- Bilateral filtering smoothed the image while preserving edges better because it used both distance and intensity similarity.
- The manual bilateral filter produced a result similar to OpenCV's bilateral filter, confirming the implementation.



Q10: Original Einstein image, Gaussian blur result, OpenCV bilateral result and manual bilateral result.

11. Relationship Between Spatial Filtering and Frequency Response

For an image f(x,y) and filter h(x,y), convolution in the spatial domain corresponds to multiplication in the frequency domain.

```
g(x,y) = f(x,y) * h(x,y)     ↔     G(u,v) = F(u,v) · H(u,v)
```

- Averaging (box) and Gaussian filters act as low-pass filters; they smooth the image and suppress high-frequency noise.
- The Gaussian filter is smoother in both domains and avoids ringing artifacts that occur with ideal low-pass filters.
- The Laplacian filter emphasizes high-frequency components, so it highlights edges but also amplifies noise.
- For high-frequency noise, a Gaussian filter is more suitable than a Laplacian filter because it suppresses high frequencies while preserving a visually natural result.

12. Illumination Correction Using Homomorphic Filtering

Non-uniform illumination was modeled as $f(x,y) = i(x,y)\, r(x,y)$, where illumination $i(x,y)$ varies slowly and reflectance $r(x,y)$ contains higher-frequency detail. By taking the logarithm, multiplication is converted to addition:

```
log f(x,y) = log i(x,y) + log r(x,y)

# Homomorphic filtering steps:
# 1) log transform  2) FFT  3) high-pass type filter
# 4) inverse FFT  5) exp transform and normalize
```

- Compared with histogram equalization, homomorphic filtering is preferred when the main problem is uneven lighting rather than only low contrast.
- Applied to the “woman at the window” image, homomorphic filtering made the indoor region and window structure more visible and balanced the bright and dark regions.
- A mild gray glow/uneven shading near the image borders was observed as an artifact.

Q12: Original image and homomorphic-filtered result.

Conclusion

The assignment demonstrated a range of intensity transformations and neighborhood filters. Gamma correction and contrast stretching improved tonal balance, histogram equalization enhanced contrast, Gaussian and bilateral filters smoothed images, and derivative-based operators highlighted edges. Bilinear interpolation outperformed nearest-neighbor interpolation in zooming, median filtering was more effective than Gaussian smoothing for salt-and-pepper noise, and homomorphic filtering improved images with non-uniform illumination. In each case, the chosen method was interpreted by comparing its visual effect and, where appropriate, objective measures such as SSD.